

Guided Practice: Image Thumbnails-GridView

Outcome

You will create and run an Android mobile app using Android Studio and the Kotlin programming language.

Resources Needed

- VCASTLE
- OR --
- System configured for Android development.

Level of Difficulty

Moderate

Deliverables

Deliverables are marked in red font or with a red picture border around the screenshot. **Your name should be visible as directed in screenshots that you submit.**

You will submit a Word document that contains screenshots that provide evidence that you have completed the tutorial.

For your submission, take screenshots as follows and place them in a Word document.

- 1) Code in MainActivity.kt
 - a) Make sure your name is visible in a comment above the class name.
 - b) This may take more than one screen capture.
- 2) Design surface of activity_main.xml
- 3) Code in ImageAdapter.kt
 - a) Make sure your name is visible in a comment above the class name.
 - b) This may take more than one screen capture.
- 4) Code in GridViewModal.kt
 - a) Make sure your name is visible in a comment above the class name.
 - b) This may take more than one screen capture.
- 5) Design surface of row_item.xml
- 6) The application in the running emulator:
 - a) Main Activity
 - b) Main Activity after clicking an image

- c) Main Activity after clicking a second image
- d) Main Activity after clicking a third image

Program Specifications

This practice program is designed to help you build an Android app that includes the incorporation of a GridView, images, GridView click event handlers, formatted text, and custom icons.

Development Steps

These development steps describe completing the work in VCASTLE. If you are using your own local installation of Android Studio, the steps may vary slightly.

Create the Android Project

- 1) From the Welcome to Android Studio window, click the New Project button.
- 2) In the New Project Template selection window, select “Phone and Tablet” in the left-hand navigation panel (should be selected by default) and select “Empty Views Activity”.
- 3) Click the “Next” button.
- 4) On the New Project Empty Activity configuration screen:
 - a) In the “Name” field, enter a name for your application (GP Gridview Images).
 - b) Leave the “Package name” field as the auto-generated value.
 - c) Leave the “Save location” field as the default value (or specify a location of your choosing).
 - d) Leave the Language field as the default value (Kotlin).
 - e) Leave the Minimum SDK field as the default value (API 24).
 - f) Leave the “Build configuration language” field as the default value (Kotlin DSL).
- 5) Click the “Finish” button to create the application.
- 6) Initializing and loading the new project may take several minutes - be patient.

Add Project Resources

String resources

- 1) In the Android Project panel, expand the “res” folder then the “values” folder to reveal the colors.xml, strings.xml, and themes.xml files.
- 2) Open the strings.xml file (double-click it).
- 3) Using techniques used in previous weeks, add string resources with the following properties:
 - a) Key: txt_eagle
Default Value: Bald Eagle
 - b) Key: txt_elephant
Default Value: Walking Elephant
 - c) Key: txt_gorilla

Default Value: Gorilla Eating

d) Key: txt_panda

Default Value: Panda Bear

e) Key: txt_panther

Default Value: Stalking Panther

f) Key: txt_polarbear

Default Value: Polar Bear and Cubs

- 4) Following the updates, the strings.xml file should appear as follows (after confirming the strings, you may close the strings.xml file):

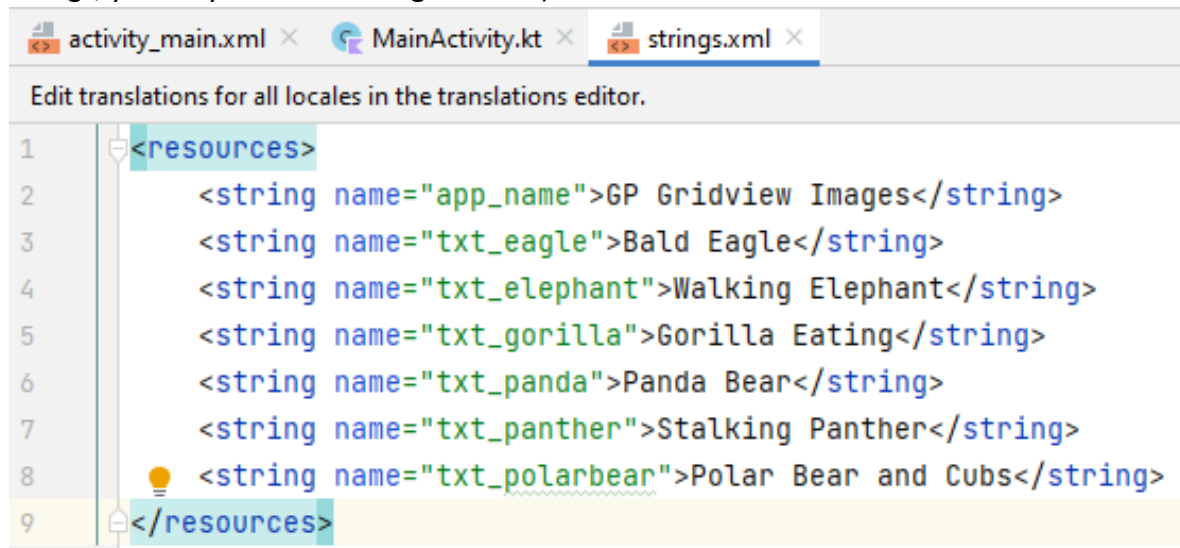


Image resources

In this section we will add image resources as we have done in previous weeks.

- 1) Expand the Resource Manager panel.
- 2) In the Resource Manager panel, click the Add resources to the module button (+ sign) to open the add menu, then click Import Drawables.
- 3) This will open the Import drawables window and the Select Path dialog. Note that only local drives will be available (i.e. drives with a letter, like the C: drive), so it is best to have the image(s) you want in an accessible location like the Pictures folder. Navigate to the location of the files, select the following files (select all of them at once), then click OK.
 - a) eagle.png
 - b) elephant.png
 - c) gorilla.png
 - d) panda.png
 - e) panther.png

- f) polar.png
- 4) This will place the files in a ready to import state. Change the names as follows, then click the Next button on the dialog.
 - a) eagle – rename to img_eagle
 - b) elephant – rename to img_elephant
 - c) gorilla – rename to img_gorilla
 - d) panda – rename to img_panda
 - e) panther – rename to img_panther
 - f) polar – rename to img_polar
- 5) This will bring up the Import drawables window, click the Import(C) button.
- 6) You should now see the image files have been added to the Resource Manager.

Icon resources

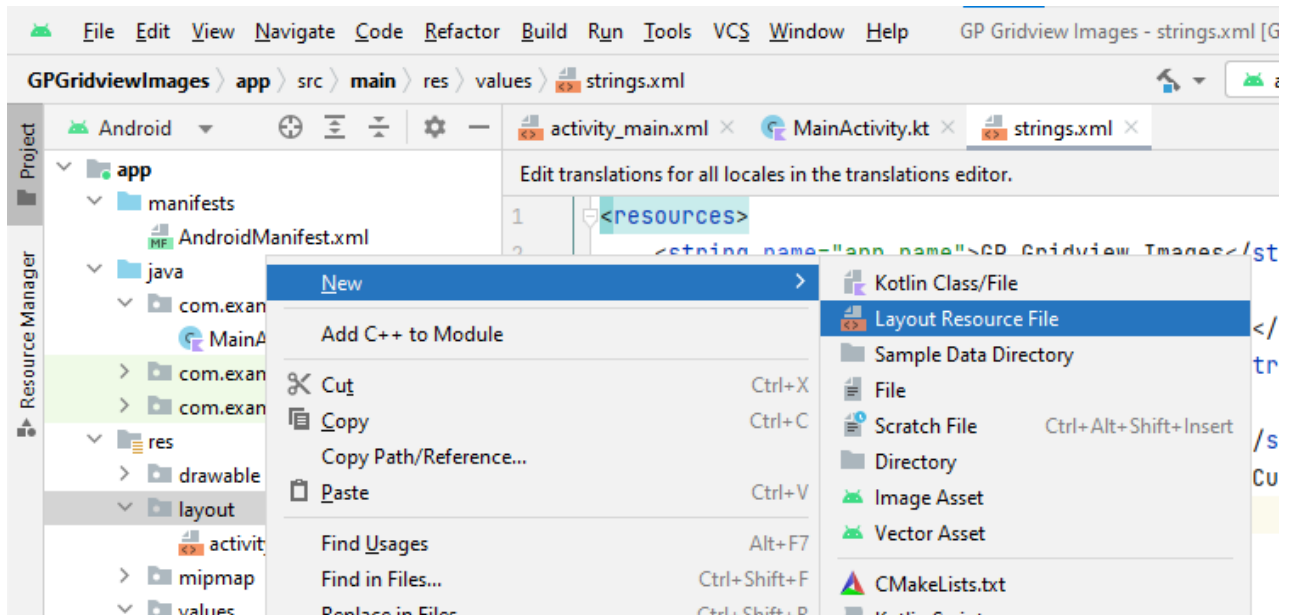
This will use the same techniques used in previous weeks.

- 1) On the left-hand navigation bar, select Resource Manager (if it isn't already open).
- 2) In the Resource Manager panel, click the Add resources to the module button (+ sign) to open the add menu, then click Image Asset.
- 3) This will open the Asset Studio window in the Configure Image Asset page. On the Configure Image Asset page, click the folder icon in the Path box.
- 4) Note that only local drives will be available (i.e. drives with a letter, like the C: drive), so it is best to have the image(s) you want in an accessible location like the Pictures folder. Navigate to the location of the gallery_icon.png file, select it, and click OK.
- 5) Resize the image to 50% to make it fit nicely in the icon safe zone.
- 6) Click Next. This will open the Confirm Icon Path page. You will see warning regarding some files being overwritten. That is nothing to be concerned about because we're replacing the original icon images.
- 7) Click Finish. This will close the window and the icon will be imported. You can now minimize the Resource Manager.

Add a layout for the images

We need a way to tell the application how we want to display our images within the grid view, so we'll create a layout for the images that will be used by the GridView. Note that this is NOT a new activity – it's just a new layout file.

- 1) In the project panel, right-click on the layout folder and select New > Layout Resource File.



- 2) The New Resource File dialog will appear. Enter row_item in the File Name field. Leave all other settings as their default values.
- 3) Click "OK".
- 4) This will create and open the row_item.xml file. Note that the new layout file already includes a Constraint Layout by default.

Add UI Components

In this section we'll build the UI for the row_item layout.

- 1) Select (click) the row_item.xml file tab in the IDE if it isn't already selected.
- 2) Switch to the Code view, and replace the code with the following:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_margin="5dp"
    app:cardCornerRadius="5dp"
    app:cardElevation="5dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <ImageView
            android:id="@+id/imgRowItem"
```

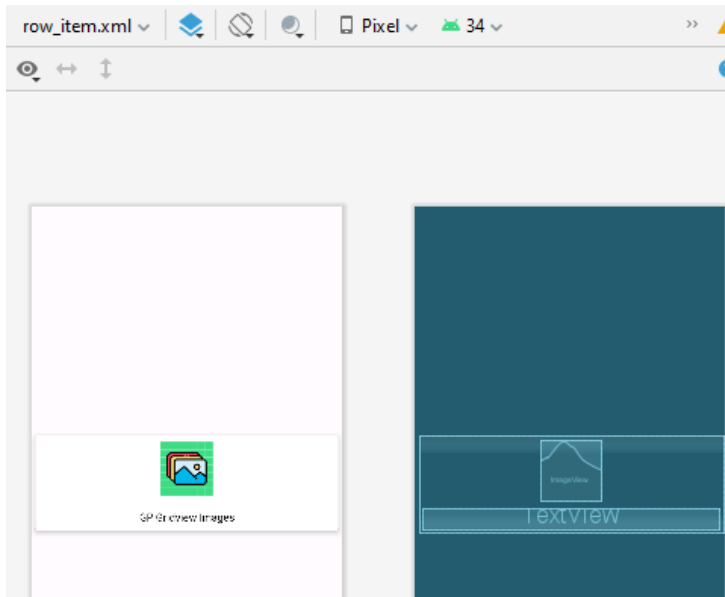
```

        android:layout_width="80dp"
        android:layout_height="80dp"
        android:layout_gravity="center"
        android:layout_margin="5dp"
        android:padding="4dp"
        android:src="@mipmap/ic_launcher" />

        <TextView
            android:id="@+id/txtRowItem"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:layout_margin="5dp"
            android:padding="4dp"
            android:text="@string/app_name"
            android:textAlignment="center"
            android:textColor="@color/black" />
    </LinearLayout>
</androidx.cardview.widget.CardView>

```

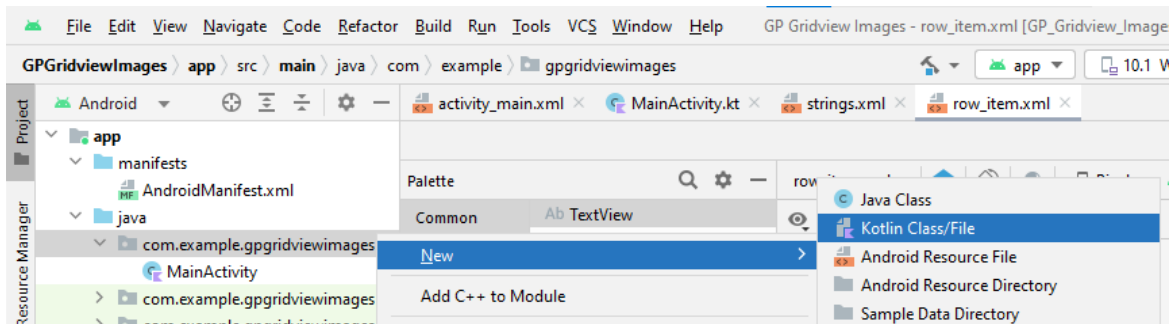
3) Following the updates, the layout should look similar to the following:



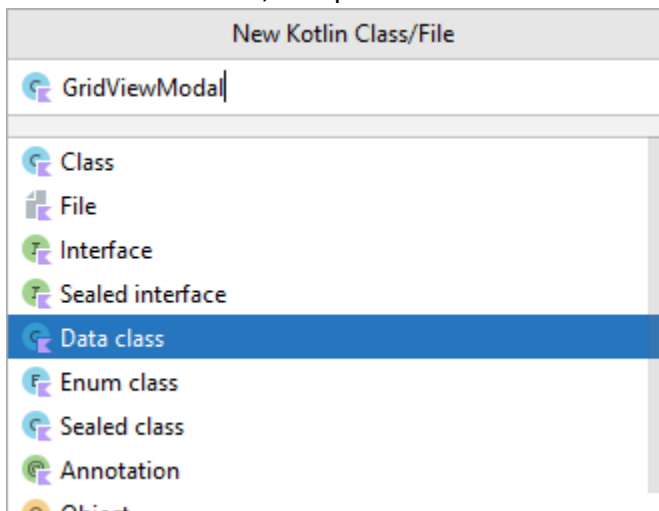
Add a data class

Now we need to create a class that can be used to hold the data associated with an image, specifically the image name and the image resource ID.

- 1) In the project panel, right-click on the com.example.gpgridviewimages folder and select New > Kotlin Class/File.



- 2) In the New Kotlin Class/File dialog, enter the name GridViewModal, select Data class in the list below the name, and press Enter.



- 3) This will create and open the ImageAdapter class file, ImageAdapter.kt. Enter the following code in the ImageAdapter.kt file. This is the entire contents of the file. Don't forget to replace the template name information with your name and the date.

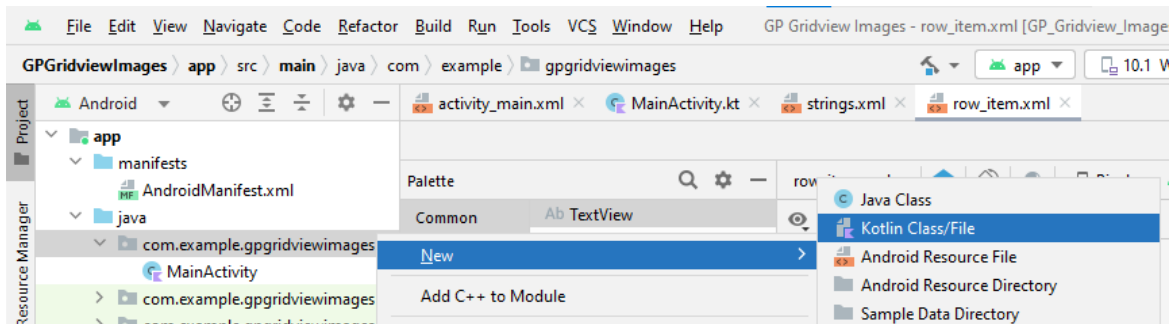
```
package com.example.gpgridviewimages

//Student Name
//GP Gridview Images
//Date
data class GridViewModal (
    val imageName: String,
    val imageId: Int
)
```

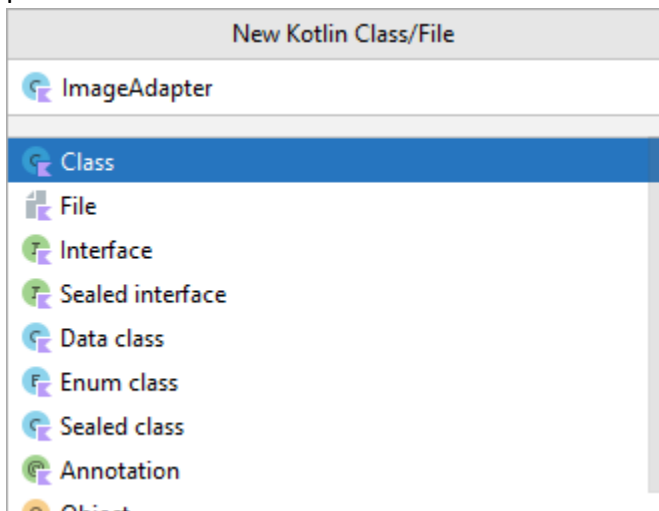
Add an image adapter class

Now we need to create a class that can be used to put an image and its associated text into the layout we just created. This is an “Adapter” class, derived from the BaseAdapter class.

- 4) In the project panel, right-click on the com.example.gpgridviewimages folder and select New > Kotlin Class/File.



- 5) In the New Kotlin Class/File dialog, enter the name ImageAdapter, leave Class selected, and press Enter.



- 6) This will create and open the ImageAdapter class file, ImageAdapter.kt. Enter the following code in the ImageAdapter.kt file. This is the entire contents of the file. Don't forget to replace the template name information with your name and the date.

```
package com.example.gpgridviewimages

import android.content.Context
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.BaseAdapter
import android.widget.ImageView
import android.widget.TextView

//Student Name
//GP Gridview Images
//Date
internal class ImageAdapter(
    private val context: Context,
    private val imagesList: List<GridViewModal>
) :
    BaseAdapter() {
    private var inflater: LayoutInflater? = null
```

```

private lateinit var imageView: ImageView
private lateinit var textView: TextView

override fun getCount(): Int {
    return imagesList.size
}

override fun getItem(position: Int): Any? {
    return null
}

override fun getItemId(position: Int): Long {
    return 0
}

override fun getView(
    position: Int,
    convertView: View?,
    parent: ViewGroup
): View? {
    var convertView = convertView
    if (layoutInflater == null) {
        layoutInflater = context.getSystemService(
            Context.LAYOUT_INFLATER_SERVICE
        ) as LayoutInflater
    }
    if (convertView == null) {
        convertView =
            layoutInflater!!.inflate(R.layout.row_item, null)
    }
    imageView = convertView!!.findViewById(R.id.imgRowItem)
    textView = convertView.findViewById(R.id.txtRowItem)
    imageView.setImageResource(imagesList.get(position).imageId)
    textView.setText(imagesList.get(position).imageName)
    return convertView
}
}

```

Update the MainActivity

Add UI Components

In this section we'll add the user interface components our app needs.

- 1) Select (click) the activity_main.xml file tab in the IDE (the file should have been opened in design view by default when the IDE opened after creating the project – if it is not open, navigate to app\res\layout and double-click activity_main.xml to open the file in design view).
- 2) Switch to the Code view, and replace the code with the following:

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

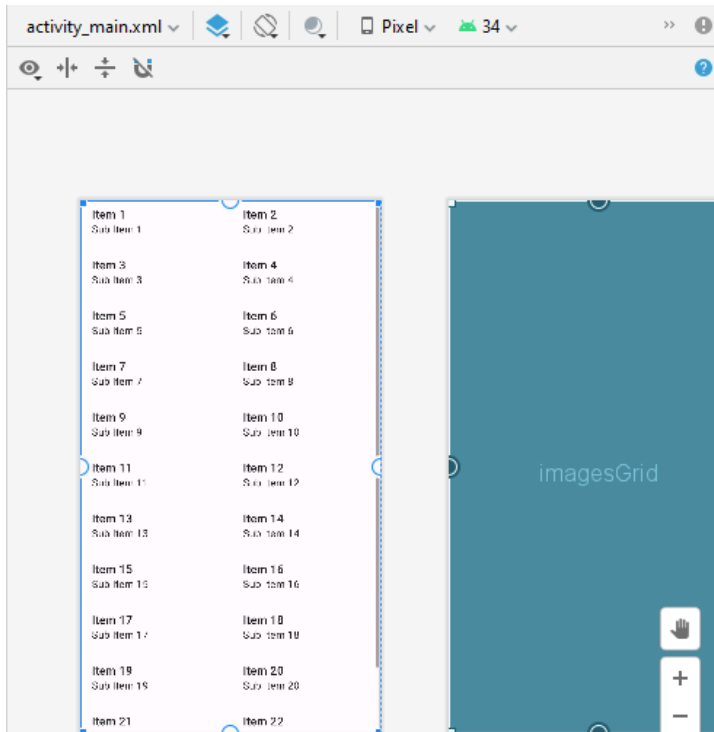
```

```

<GridView
    android:id="@+id/imagesGrid"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:horizontalSpacing="6dp"
    android:numColumns="2"
    android:verticalSpacing="6dp" />
</RelativeLayout>

```

3) In the end, the activity_main design view should look similar to the following:



Update MainActivity.kt code

Now that we have our main activity layout, we'll add code to set up and display our images and their titles.

- 1) Click the tab for the MainActivity.kt file (or open the file if it was closed).
- 2) Immediately above the class declaration, add a comment with your name, the assignment name, and the date.
- 3) Enter the following code in the MainActivity.kt file. This is the entire contents of the file. Don't forget to replace the template name information with your name and the date.

```

package com.example.gpgridviewimages

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.AdapterView
import android.widget.GridView
import android.widget.Toast

```

```

//Student Name
//GP Gridview Images
//Date
class MainActivity : AppCompatActivity() {
    lateinit var imgGrid: GridView
    lateinit var imagesList: List<GridViewModal>

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        imgGrid = findViewById(R.id.imagesGrid)

        //Add the images and titles to the list
        imagesList = ArrayList<GridViewModal>()
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_eagle), R.drawable.img_eagle)
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_elephant), R.drawable.img_elephant)
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_gorilla), R.drawable.img_gorilla)
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_panda), R.drawable.img_panda)
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_panther), R.drawable.img_panther)
        imagesList = imagesList + GridViewModal(
            getString(R.string.txt_polarbear), R.drawable.img_polar)

        val imgAdapter =
            ImageAdapter(this@MainActivity, imagesList)
        imgGrid.adapter = imgAdapter
        imgGrid.setOnItemClickListener =
            AdapterView.OnItemClickListener { _, _, position, _ ->
                Toast.makeText(
                    applicationContext,
                    "You CLicked " + imagesList[+position].imageName,
                    Toast.LENGTH_LONG).show()
            }
    }
}

```